

Navigate with a Rover Base

The [navigation service](#) allows you to queue up user-defined waypoints for your machine to move to in the order that you specify. You can also add obstacles or set linear and angular velocity targets in your navigation service config. Viam’s motion planner will plan routes that avoid those obstacles and attempt to keep the robot at your specified velocity.

Learning Goals

After following this tutorial, you will be able to:

- add waypoints to specify locations you want your machine to move to, both in the app and programmatically
- use the navigation service to move your machine along those waypoints

To try it out yourself, you need a mobile base and a movement sensor that can track the robot’s GPS coordinates and angular and linear velocity. Follow this tutorial to get started using Viam’s Navigation service to help your wheeled base navigate across space with our recommended stack.



Requirements

1 **A base**

We used [a LEO rover](#), configured as a [wheeled base](#), but you can use whatever model of rover base you have on hand:



2 A movement sensor with GPS position, compass heading, and angular and linear velocity readings

We used three movement sensors to satisfy these requirements:

1. A [SparkFun GPS-RTK-SMA Breakout movement sensor](#) configured as a `gps-nmea-rtk-serial` model, providing GPS position and compass heading measurements.
2. A [wheeled-odometry](#) model gathering angular and linear velocity information from the [encoders](#) wired to our base's [motors](#).
3. A [merged](#) model aggregating the readings together for the navigation service to consume.

You can use any combo of movement sensors you want as long as you are getting all the types of measurements required. See [the navigation service](#) for more info on movement sensor requirements.

Tip

If you are using different hardware, the navigation setup process will be mostly the same, but you will need to substitute your robot's components.

Before you start, make sure to create a machine in the [Viam app](#) and [install viam-server](#) on your robot. Also, make sure to physically connect your components together to your machine's computer and power it on.

Configure the components you need

First, configure the `components` of your robot. If you are using different hardware, configure them according to the instructions for the component model that supports your hardware.

> Click to see how we configured our LEO rover

Configure movement sensors

[Config Builder](#) [JSON](#)

- 1 Configure a GPS movement sensor so the robot knows where it is while navigating. We configured ours as a `gps-nmea-rtk-serial` movement sensor:

gps

movement_sensor

gps-nmea-rtk-serial

```

1  {
2    "ntrip_password": "yourpassword",
3    "ntrip_url": "http://your.url:8082",
4    "ntrip_username": "yourusername",
5    "serial_baud_rate": 115200,
6    "serial_path": "/dev/serial/by-id/usb-u-blox_AG_-_www.u-blox.com_u-blox_GNSS_recei
7    "ntrip_connect_attempts": 10,
8    "ntrip_mountpoint": "NJI2"
9  }

```

Depends on

Search resources

Data capture

+ Add method

We named ours `gps`. Refer to [the `gps-nmea-rtk-serial` movement sensor documentation](#) for attribute information.

- Configure a wheeled odometry movement sensor to provide angular and linear velocity measurements from the encoded motors on our base.

enc-linear

movement_sensor

wheeled-odometry

Attributes

base *

base

left_motors *

left-motor ×

right_motors *

right-motor ×

Show more

Depends on

Search resources

Data capture

+ Add method

We named ours `enc-linear`. Refer to [the `wheeled-odometry` movement sensor documentation](#) for attribute information.

- Now that you've got movement sensors which can give you GPS position and linear and angular velocity readings, configure a `merged` movement sensor to aggregate the readings from our other movement sensors into a singular sensor:

merged

movement_sensor

merged

```

1  {
2    "angular_velocity": [
3      "enc-linear"
4    ],
5    "compass_heading": [
6      "gps"
7    ],
8    "linear_acceleration": [
9      "enc-linear"
10   ],
11   "linear_velocity": [
12     "enc-linear"
13   ],
14   "orientation": [
15     "gps"
16   ],
17   "position": [
18     "gps"
19   ]
20 }

```

Depends on

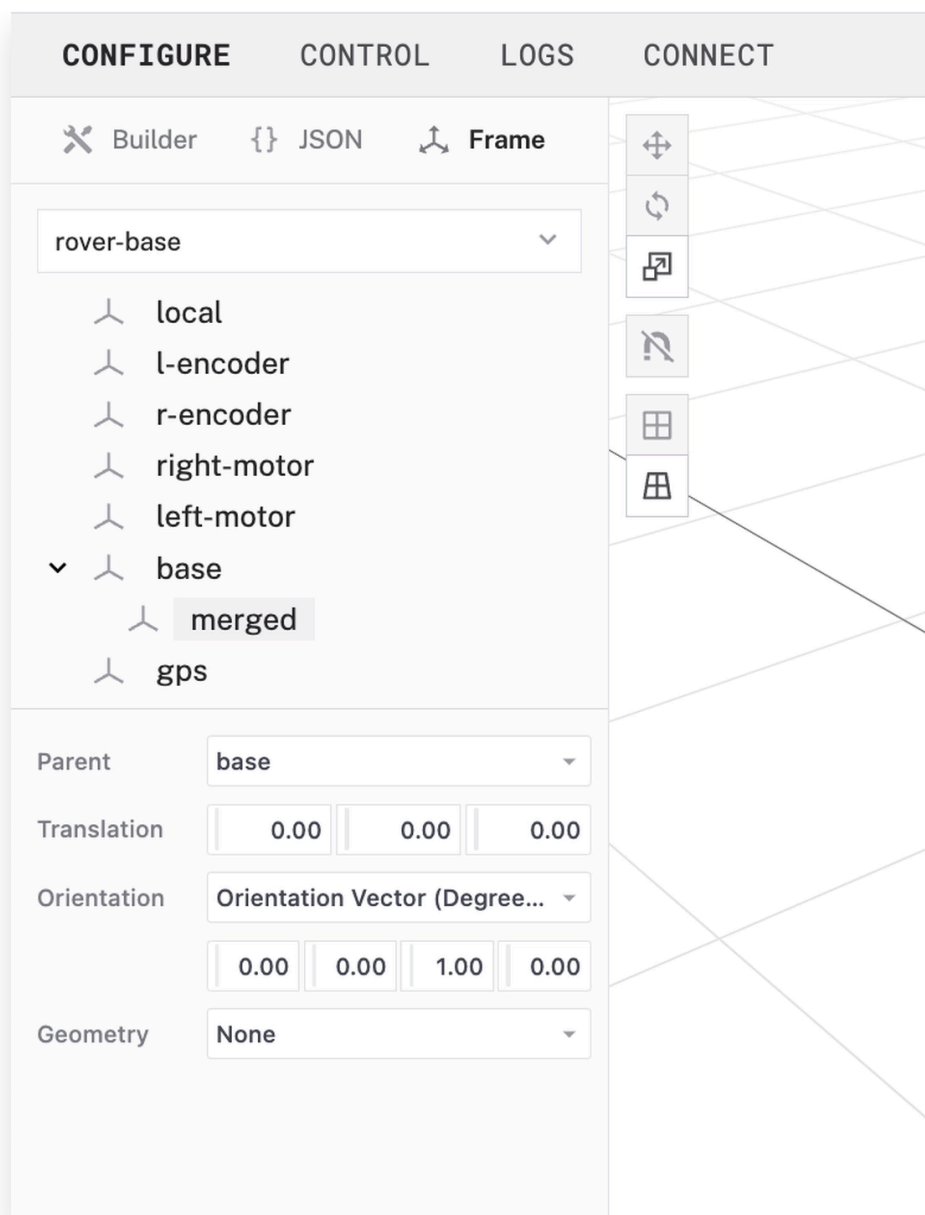
Search resources

Data capture

+ Add method

We named ours `merged`. Refer to [the `merged` movement sensor documentation](#) for attribute information.

- Make sure your `merged` movement sensor is configured to gather `"position"` readings from the `gps` movement sensor.
- [Configure the frame system](#) for this movement sensor so that the navigation service knows where it is in relation to the base.
 - Switch to **Frame** mode on the **CONFIGURE** tab and select your movement sensor. If your movement sensor is mounted on top of the rover like ours is, set **Orientation**'s third input field, Z, to `1`.
 - Select the `base` as the parent frame.



Configure a navigation service

Config Builder JSON

Add the navigation service so that your wheeled base can navigate between waypoints and avoid obstacles. To add the navigation service to your robot, do the following:

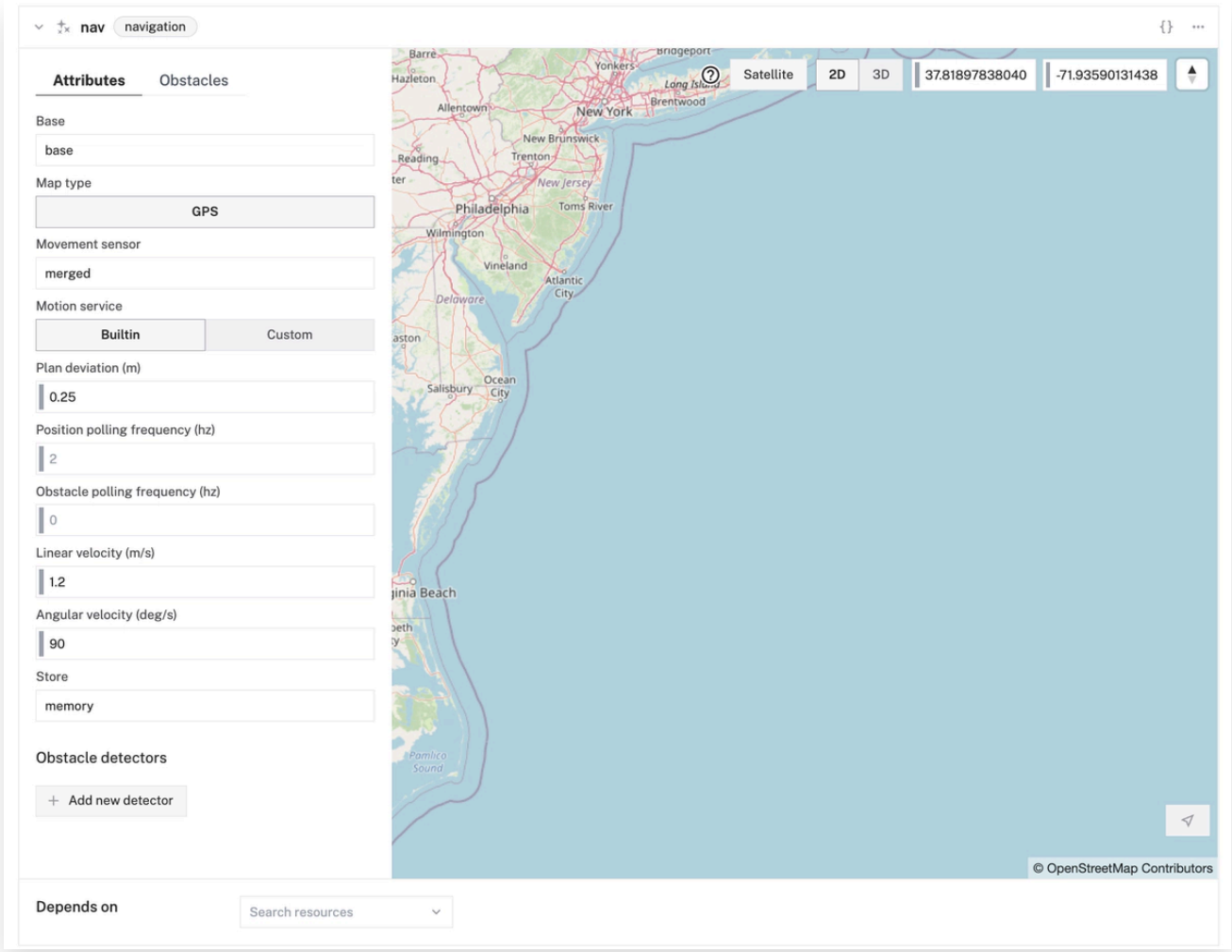
- 1 Navigate to the **CONFIGURE** tab of your machine's page in the [Viam app](#).
- 2 Click the + icon next to your machine part in the left-hand menu and select **Service**.
- 3 Select the **navigation** type.
- 4 Enter a name or use the suggested name for your service and click **Create**.
- 5 Select **JSON** mode. Copy and paste the following into your new service's attributes field:

```
{
  "base": "base",
  "movement_sensor": "merged",
  "obstacles": [],
  "store": {
    "type": "memory"
  },
  "position_polling_frequency": 2,
  "meters_per_sec": 1.2,
  "degs_per_sec": 90,
  "plan_deviation_m": 0.25
}
```

Edit the attributes as applicable. Attribute information is available in [the navigation service documentation](#).

- 6 Click **Save** in the top right corner of the screen to save your changes.

Your navigation service should now appear in your machine's **CONFIGURE** tab as a card with a map like the following:



For more detailed information see [the navigation service](#).

Start navigating with the navigation service

Now that you have configured your navigation service, add waypoints to your navigation service. You can add waypoints from the [CONTROL tab](#) or [programmatically](#).

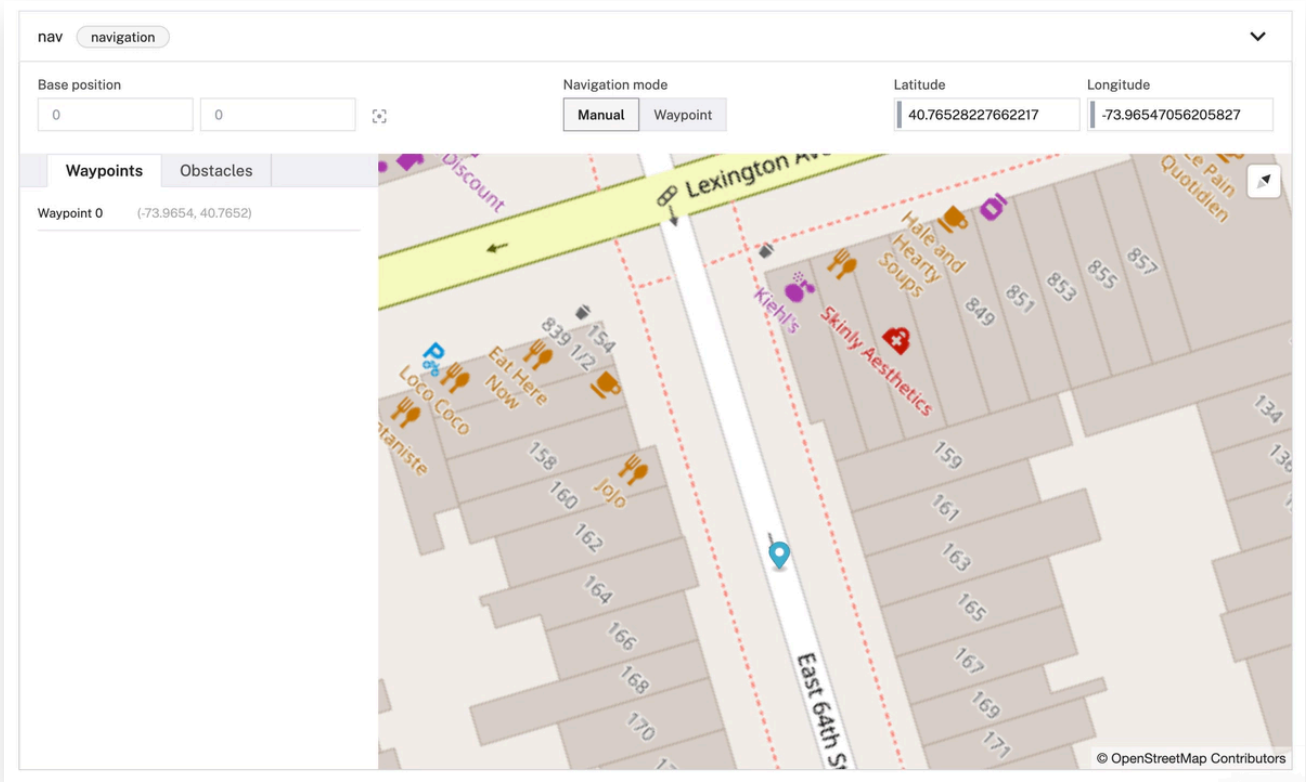
Control tab method

Go to the **CONTROL** tab of your robot in the [Viam app](#), and open the navigation service card.

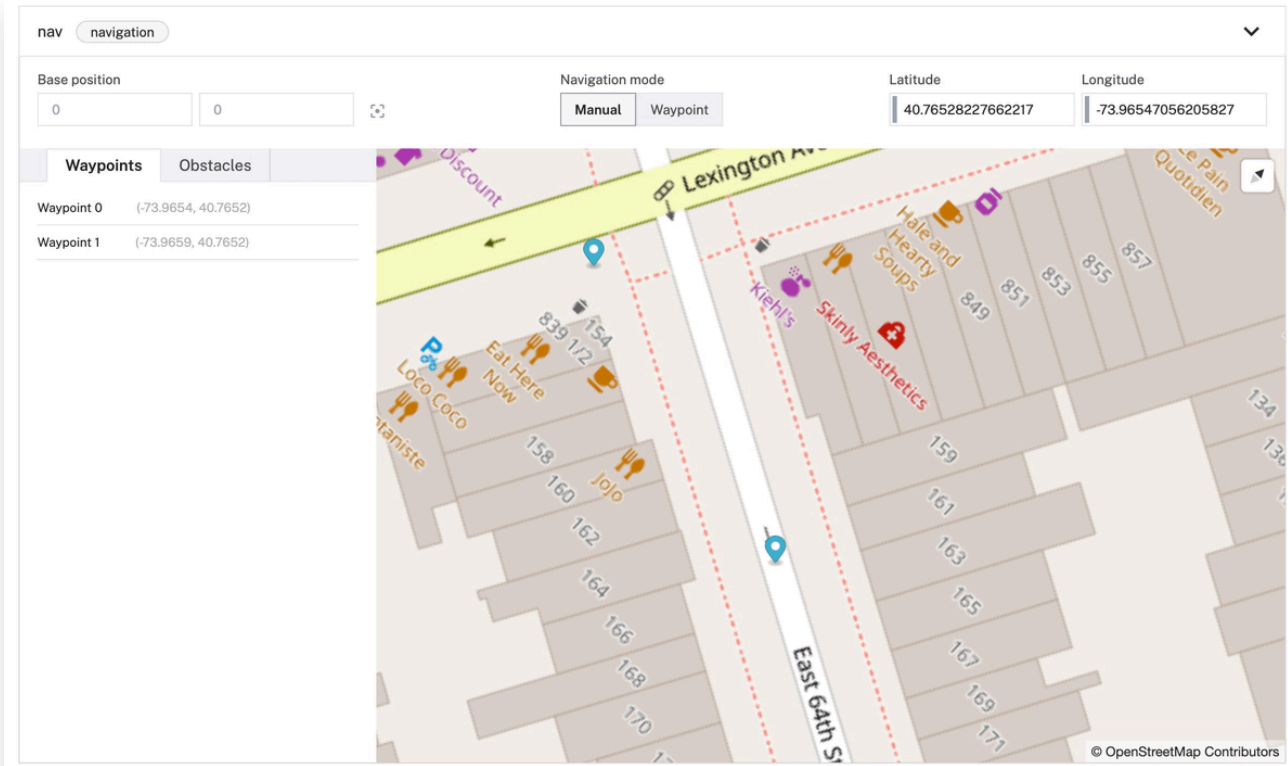
From there, ensure that **Navigation mode** is selected as **Manual**, so your robot will not begin navigation while you add waypoints.

Add waypoints

Select **Waypoints** on the upper-left corner menu of the navigation card. Zoom in on your current location and click on the map to add a waypoint.



Add as many waypoints as you desire. Hover over a waypoint in the left-hand menu and click the trash icon to delete a waypoint.



(Optional) Add obstacles

If you want your robot to avoid certain obstacles in its path while navigating, you can also add obstacles. In the **CONFIGURE** tab, select the **Obstacles** subtab on the navigation card. Zoom in on your current location, then hold **shift** and drag on the map to draw an obstacle. Add as many obstacles as you desire. Hover over an obstacle in the left-hand menu and click the trash icon to delete an obstacle.

Begin navigation

Toggle **Navigation mode** to **Waypoint**. Your rover will begin navigating between waypoints.



Programmatic method

If you want to do add waypoints programmatically, use the service's [API method](#) `AddWaypoint()` :

Add waypoints

```
myNav, err := navigation.FromRobot(robot, "my_nav_service")

// Create a new waypoint at the specified latitude and longitude
location = geo.NewPoint(40.76275, -73.96)

// Add your waypoint to the service's data storage
err := myNav.AddWaypoint(context.Background(), location, nil)
```

Begin navigation

To start navigating, set your service to `MODE_WAYPOINT` with the service's [API method](#) `SetMode()` :

[Go](#) [Python](#)

```
myNav, err := navigation.FromRobot(robot, "my_nav_service")

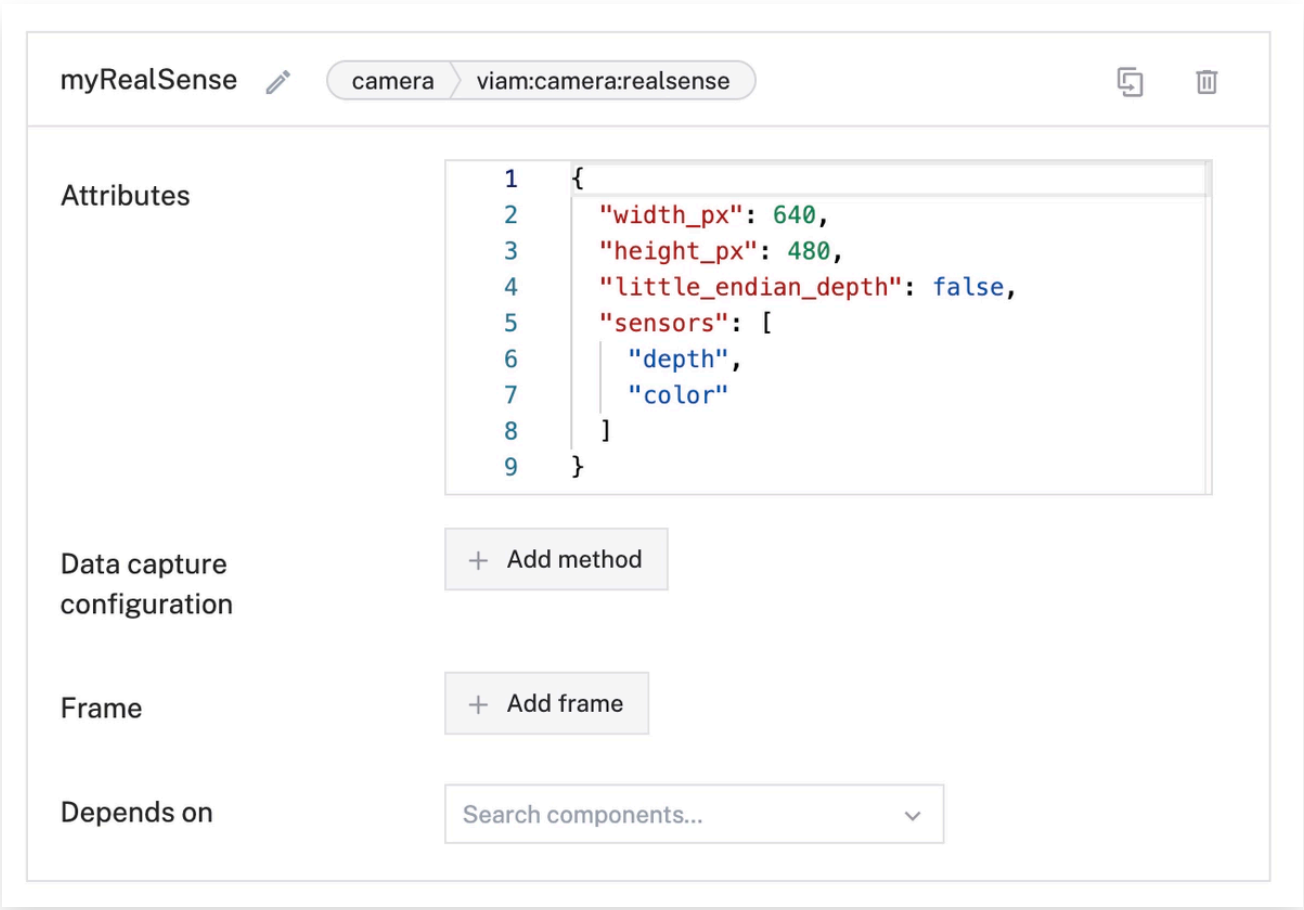
// Set the Mode the service is operating in to MODE_WAYPOINT and begin navigation
mode, err := myNav.SetMode(context.Background(), Mode.MODE_WAYPOINT, nil)
```

Next steps: automate obstacle detection

In this tutorial, you have learned how to use Navigation to navigate across waypoints. Now, you can make navigation even better with automated obstacle detection.

First, configure a depth [camera](#) that your robot can sense how far away it is from obstacles.

We configured ours as an Intel RealSense Camera, which is available as a [modular resource](#) [in the Viam Registry](#):



Tip

You can alternatively use `viam:ultrasonic:camera` from the official viam module.

If you want the robot to be able to automatically detect obstacles in front of it, [configure a Vision service segmenter](#). For example, [configure](#) the Vision service model `obstacles_depth` to detect obstacles in front of the robot. Then, use one of [Viam's client SDKs](#) to automate obstacle avoidance with the navigation service like in the following Python program:

➤ [Click to view full example of automated obstacle avoidance with the Python SDK](#)

You can also ask questions in the [Community Discord](#) and we will be happy to help.

 Ask AI